1. Domínio da aplicação:

A linguagem tem como domínio principal o desenvolvimento de APIs, especificamente serviços HTTP de backend expostos como endpoints. Essa escolha se relaciona diretamente com alguns fatores: os problemas a resolver que são recorrentes em sistemas distribuídos, como roteamento de requisições, validação e transformação de dados estruturados. Alinhado a isso, foram escolhidas como primitivas da linguagem coisas como endpoints anônimos com decorators e a palavra-chave de roteamento end. O público-alvo para essa linguagem é o desenvolvedor backend que visa previsibilidade e segurança de tipos em ambientes de produção. Contribuindo para esses propósitos, foram escolhidas a tipagem estática e a compilação. Além disso, os dados manipulados envolvem coleções estruturadas e valores potencialmente ausentes, o que justifica o suporte nativo a Optional/result e a operadores de alta ordem (map, filter, sum, order, flatten) para transformação declarativa desses dados.

2. Critérios de Avaliação:

Legibilidade (readability) é a facilidade com que um programa pode ser lido e compreendido, influenciada pela simplicidade do conjunto de construções, pela ortogonalidade (poucas regras combinadas de forma consistente), pela adequação dos tipos de dados e estruturas de controle, e pelo projeto sintático.

Escrevibilidade (writability) mede a facilidade de usar a linguagem para produzir programas corretos, dependendo da simplicidade ou ortogonalidade, do suporte a abstração e da expressividade.

Confiabilidade (reliability) refere-se ao grau em que um programa se comporta conforme sua especificação, dependendo de verificação de tipos, tratamento de exceções, aliasing e da própria legibilidade e escrevibilidade.

Custo (cost) agrega os custos de treinamento de programadores, escrita, compilação, execução, manutenção e das falhas em produção ao longo de todo o ciclo de vida da linguagem.

Em termos de legibilidade, a tipagem estática explícita e o pattern matching que também decidimos adotar tornam a intenção de cada endpoint evidente, enquanto a ausência de null remove uma fonte clássica de ambiguidade de leitura. Quanto à escrevibilidade, operadores de alta ordem (map, filter, sum, order, flatten) e decorators para endpoints reduzem o código Repetitivo típico de APIs; em contrapartida, o uso de Result/Optional exige tratamento explícito de ausência de valor, o que aumenta a verbosidade em relação a linguagens com exceções implícitas. Já a confiabilidade é fortemente favorecida pela tipagem estática (checagem em tempo de compilação), pela imutabilidade por padrão e pela eliminação de null via Result/Optional, que atacam diretamente as três maiores fontes de defeitos em APIs: erros de tipo, efeitos colaterais inesperados e NullPointerException. Por fim, em relação ao custo, a compilação estática eleva o custo de build, mas reduz o custo de execução e de manutenção ao longo do tempo, já que resulta em menos falhas em produção; a curva de aprendizado exigida por Result/Optional e pela imutabilidade é o principal custo de treinamento assumido pelo grupo.

3. **Influências e Paradigmas:**

Imperativo: baseado no modelo de von Neumann, o programa é uma sequência de comandos que altera um estado (variáveis/memória) por meio de atribuição, laços e desvios condicionais.

Funcional: computação tratada como avaliação de funções matemáticas, sem estado mutável nem efeitos colaterais; ênfase em recursão, funções de alta ordem e imutabilidade.

Lógico (declarativo): o programa é um conjunto de fatos e regras lógicas, e a execução consiste em consultas resolvidas por um motor de inferência; o programador descreve “o quê”, não “como”.

Orientado a Objetos: organiza dados e comportamento em objetos, instâncias de classes, com encapsulamento, herança e polimorfismo como pilares para reuso e extensibilidade.

A linguagem se alinha ao paradigma imperativo porque seu modelo de execução é centrado em procedimentos que executam sequências de comandos sobre um estado, com estruturas de controle explícitas como if, for, foreach e pattern matching que decidem o fluxo de execução a cada requisição roteada por um endpoint. Ainda assim, o projeto incorpora influências do paradigma funcional (variáveis imutáveis por padrão, operadores de alta ordem como map, filter, sum, order e flatten e o uso de Result/Optional no lugar de exceções para erros esperados), o que é uma tendência comum em linguagens imperativas mais novas que buscam a previsibilidade funcional sem abandonar o modelo imperativo de execução sequencial e orientada a estado, adequado ao ciclo requisição-resposta de uma API.

4. **Trade-offs:**

Escrevibilidade x confiabilidade: mais liberdade sintática e semântica facilita escrever, mas abre espaço para erros.

Legibilidade x escrevibilidade: notações muito compactas favorecem quem escreve, mas dificultam a leitura por terceiros.

Desempenho de execução x verificações de segurança: checagens em tempo de execução aumentam a confiabilidade, mas custam desempenho.

Generalidade x especificidade: construções genéricas reaproveitam-se em vários domínios, mas construções específicas de domínio tendem a ser mais concisas e seguras nesse domínio.

As decisões de projeto tomadas nesta etapa priorizam de forma consistente confiabilidade e legibilidade em detrimento de flexibilidade e escrevibilidade irrestrita, escolha coerente com um domínio no qual falhas têm custo alto. A combinação de tipagem estática com compilação prioriza confiabilidade e desempenho em tempo de execução, relativizando a escrevibilidade e a velocidade de iteração, já que o ciclo de edição-compilação-teste é mais longo do que em linguagens interpretadas. A ausência de null, substituída por Result e Optional, também prioriza confiabilidade, mas, em contrapartida, relativiza a escrevibilidade, pois o tratamento explícito da ausência de valor exige mais código do que o uso implícito

de null. Escolhendo não permitir herança (ou permitir no máximo um nível), prioriza-se simplicidade e legibilidade, mantendo hierarquias rasas e fáceis de acompanhar, ao custo da generalidade e do reuso via polimorfismo profundo, comuns em linguagens orientadas a objetos clássicas. Variáveis imutáveis por padrão favorecem a confiabilidade, reduzindo efeitos colaterais e condições de corrida em serviços concorrentes, mas relativizam a escrevibilidade em algoritmos naturalmente iterativos, que passam a exigir reatribuição por meio de novos bindings em vez de mutação direta. Da mesma forma, procedimentos privados por padrão priorizam confiabilidade e encapsulamento, protegendo por padrão a superfície interna da API, ainda que isso relativize a conveniência inicial de escrita, já que expor um procedimento exige uma palavra-chave explícita. Por fim, endpoints anônimos com decorators priorizam a escrevibilidade específica do domínio, relativizando a generalidade, uma vez que esse recurso é otimizado para roteamento HTTP e não se estende bem a outros contextos de uso.

5. **Métodos de Implementação:**

> A especificação de uma linguagem define, de forma formal ou semiformal, sua sintaxe e sua semântica. É uma descrição abstrata do que a linguagem é, independente de qualquer software. A implementação, por sua vez, é o software concreto (compilador, interpretador ou ambiente híbrido) que realiza essa especificação, traduzindo ou executando programas escritos na linguagem em uma máquina real.

Interpretação: o código-fonte (ou uma representação próxima dele) é lido e executado diretamente por um interpretador, instrução a instrução, sem gerar previamente código de máquina, o que gera maior flexibilidade e portabilidade, menor desempenho.

Compilação: um compilador traduz todo o programa-fonte para código de máquina antes da execução; o programa resultante roda diretamente no processador-alvo, com desempenho superior, mas perde portabilidade imediata entre arquiteturas.

Híbrida: combina as duas abordagens; o código-fonte é compilado para uma representação intermediária (bytecode), que é então interpretada por uma máquina virtual e/ou compilada just-in-time (JIT) para código nativo em tempo de execução, equilibrando portabilidade e desempenho.

Para esta linguagem, o método de implementação mais adequado é a compilação para código nativo (ahead-of-time), e não um modelo interpretado ou híbrido baseado em máquina virtual. O domínio de APIs de backend impõe exigências específicas que favorecem essa escolha: previsibilidade de desempenho sob carga, baixo tempo de inicialização (importante em ambientes de contêineres e funções serverless, onde a aplicação pode ser instanciada e finalizada com frequência) e a possibilidade de distribuir o serviço como um único binário autocontido, sem depender de uma máquina virtual ou runtime instalado no ambiente de destino. Além disso, como a linguagem já assume tipagem estática, imutabilidade por padrão e ausência de null via Result/Optional, a compilação ahead-of-time permite que todas essas garantias sejam verificadas integralmente antes da execução, sem custo algum em tempo de execução, o compilador realiza a checagem de tipos, a análise de imutabilidade e a cobertura de casos em pattern matching (por exemplo, garantindo que tanto o caminho de sucesso quanto o de erro de um Result sejam tratados) como parte do processo de build, reforçando exatamente os critérios de confiabilidade e desempenho priorizados no projeto da linguagem.

6. **Adoção e Longevidade de Linguagens de Programação:**

COBOL e FORTRAN permanecem em uso mesmo décadas após sua criação por uma combinação de fatores: existe um volume gigantesco de sistemas legados críticos escritos nessas linguagens, e reescrevê-los implica custo e risco altíssimos; ambas continuam sendo mantidas e atualizadas por comitês de padronização como COBOL 2014 e Fortran 2018 incorporaram recursos modernos preservando compatibilidade retroativa; o domínio de aplicação original (processamento transacional em lote para COBOL, computação numérica de alto desempenho para Fortran) permanece relevante e essas linguagens ainda atendem bem a ele.

Estratégias para garantir longevidade:

- Definir uma política clara de versionamento e compatibilidade retroativa, evitando quebras não sinalizadas entre versões (versionamento semântico).

- Investir em ferramental (compilador, formatador, testador, gerenciador de pacotes) e documentação de qualidade desde as primeiras versões, reduzindo o custo de adoção.

- Garantir interoperabilidade com o ecossistema de APIs já consolidado (JSON, OpenAPI/Swagger, protocolos HTTP/gRPC), evitando isolamento tecnológico.

- Fomentar uma comunidade e uma governança aberta (RFCs, repositório público, canais de contribuição), o que historicamente sustenta linguagens além do ciclo de vida de uma única organização.

- Priorizar estabilidade e evolução incremental do núcleo da linguagem, introduzindo novidades por meio de bibliotecas/extensões antes de alterar a linguagem-base.